

Multimedia Data Modeling

Hoa Dinh Nguyen, PhD.

hoand@ptit.edu.vn

Role of Multimedia Database Modeling

- ▶ Accurate representation of complex multimedia data structures
- ▶ Support for diverse and continuously evolving data types
- ▶ Capture strong relationships among multimedia components
- ▶ Enable efficient storage and information retrieval

Lecture Roadmap: Multimedia Data Models

- ▶ Progression: Relational → Object-Oriented → Object-Relational → NoSQL → Graph → Vector
- ▶ Each model addresses a different weakness of the previous one for multimedia data
- ▶ Relational: mature SQL; weak for nested media structure and behavior
- ▶ OO / OR: objects, inheritance, UDTs — better for complex MM entities
- ▶ NoSQL: flexible documents, scale-out for large media catalogs
- ▶ Graph: relationships and navigation;
- ▶ Vector: similarity search on embeddings



Multimedia Data presentation

- ▶ Multimedia data combines multiple media types: text, image, audio, video, animation, 3D models
- ▶ Composite objects: a web page (HTML + images + scripts), a video lecture (stream + slides + transcript)
- ▶ Rich descriptive metadata: author, creation date, format, codec, resolution, duration, GPS, keywords
- ▶ Temporal and spatial dimensions: video/audio are time-dependent; images/maps are space-dependent
- ▶ Semantic content: low-level features (color, textures) vs high-level concepts (scene, speaker, topic)



Characteristics Relevant to Data Modeling

- ▶ Volume and complexity: large BLOBs plus descriptive metadata and structural relationships
- ▶ Heterogeneity: different encoding formats, schemas, and access patterns within one application
- ▶ Structural nesting: documents contain chapters, scenes, frames, or linked sub-objects
- ▶ Evolving schemas: new formats (HEIC, AV1) and attributes appear over the system lifetime
- ▶ Cross-media relationships: hyperlink, embedding, synchronization, version, and derivation links
- ▶ Implication: the data model must represent structure, behavior, and relationships uniformly



Relational Data Model

- ▶ Data represented in flat table structures
- ▶ Limited ability to express complex data relationships
- ▶ Static and strictly defined inter-table relationships

Relational Data Model

▶ Limitations

- ▶ Difficulty representing complex multimedia structures
- ▶ Limited flexibility in schema evolution
- ▶ Strict integrity constraints across tables

Relational Model vs Multimedia: A Concrete Example

- ▶ Scenario: store an online course video with chapters, subtitles, thumbnail, and instructor profile
- ▶ Relational design: Video, Chapter, Subtitle, Thumbnail, Instructor, Video_Instructor (6+ tables)
- ▶ Problems: many JOINS for one logical object; subtitle timing stored awkwardly; nested order implicit
- ▶ Behavior (extract frame, generate preview) lives outside the DB in application codes
- ▶ Schema change (add 360° stream) requires new tables and migration across related rows



Example — Relational Schema Analysis

- ▶ Design a relational schema for a photo-sharing app storing: photos, EXIF metadata, user tags, albums, and comments
- ▶ Draw tables with primary/foreign keys (on paper or using any ER tool)
- ▶ Questions:
 - ▶ (a) How many tables and JOINS are needed to retrieve one photo with all tags and comments?
 - ▶ (b) How do you represent a photo that belongs to multiple albums?
 - ▶ (c) What difficulties arise when adding video clips with the same tagging model?
- ▶ Discuss limitations before moving to the object-oriented model



Object Oriented Database Model

- ▶ Data is organized in three parts
 - ▶ Objects: encapsulate data and behavior
 - ▶ Classes: define attributes and associated methods.
 - ▶ Class hierarchies: includes connection structures between classes.

Why Object-Oriented Databases for Multimedia?

- ▶ Objects map naturally to multimedia entities: Document, Image, VideoClip, Museum, Scene
- ▶ Encapsulation: data (attributes) and behavior (methods) are stored together in one unit
- ▶ Object identity (OID): stable reference even when physical storage location or format changes
- ▶ Complex and user-defined types: arrays, sets, structures, BLOB references as first-class attributes
- ▶ Inheritance: share common attributes/methods (e.g., FindDocs) while specializing per media type
- ▶ Better match for application objects in Java/C++/Python multimedia frameworks

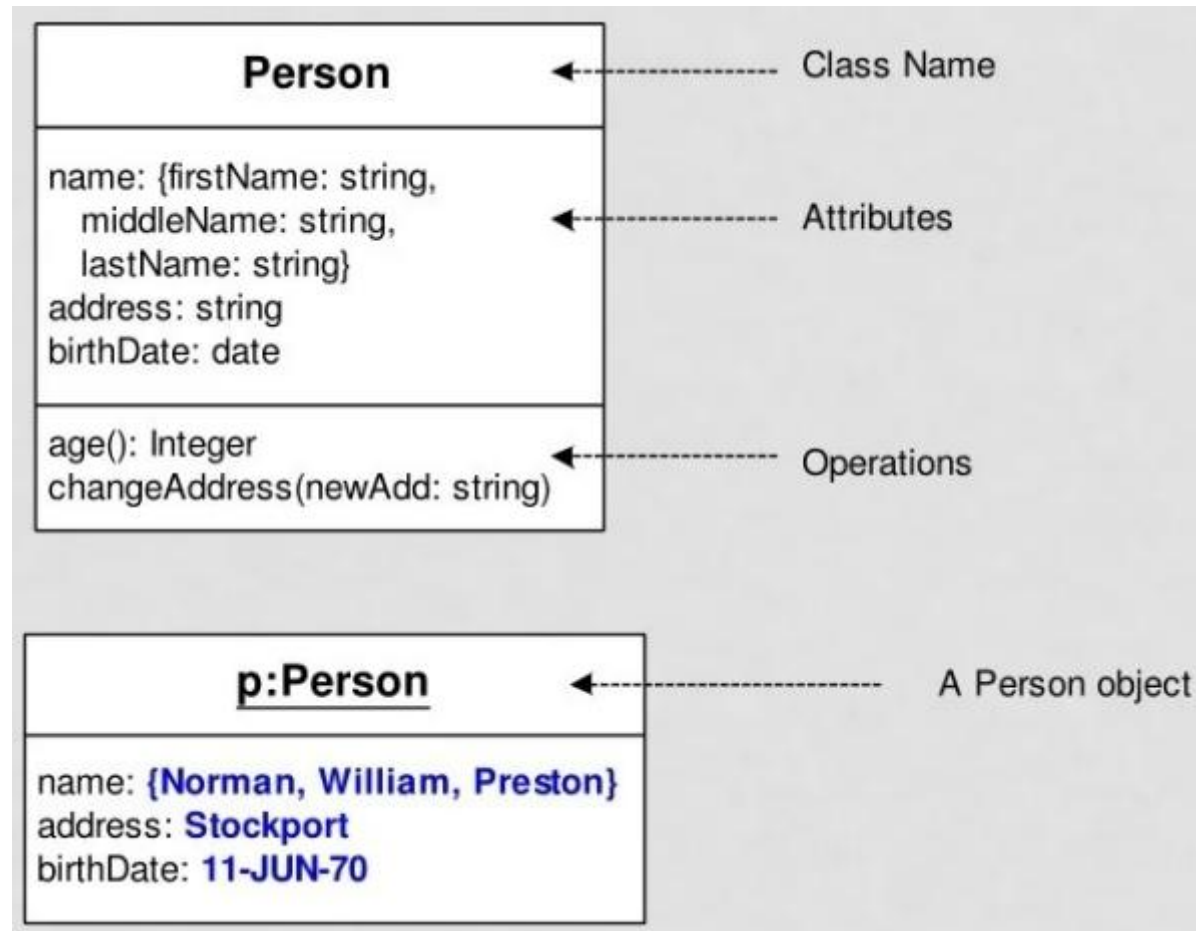


Core OODB Concepts

- ▶ Object: instance with unique OID, attribute values, and applicable methods
- ▶ Class: template defining attributes, methods, and constraints for a set of objects
- ▶ Class hierarchy: specialization/generalization (is-a); subclass inherits parent attributes and methods
- ▶ Aggregation (has-a): object contains or references other objects (document → images, links)
- ▶ Polymorphism: same method name behaves differently depending on the object's class
- ▶ Persistence: objects outlive the program that created them; DBMS handles storage and retrieval

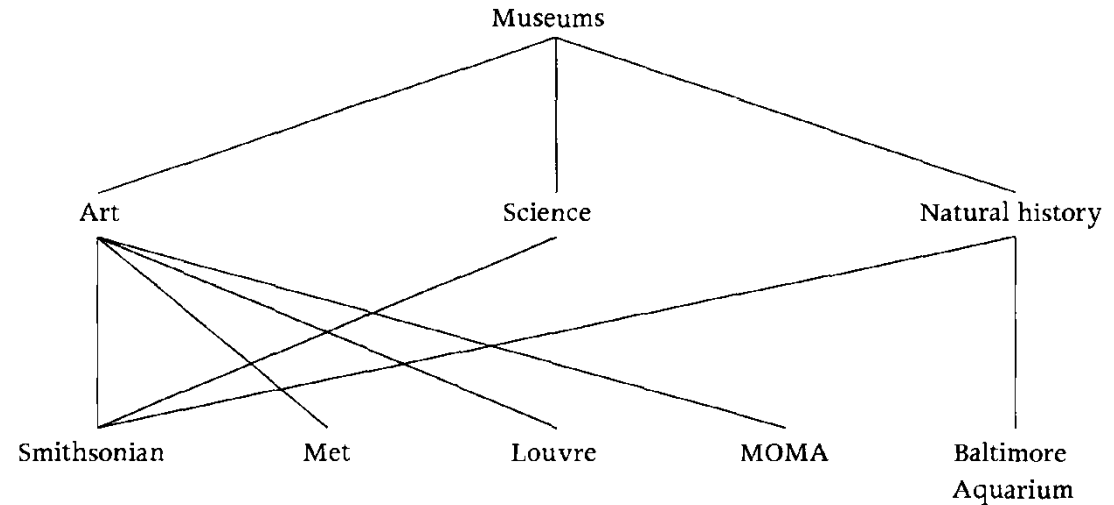
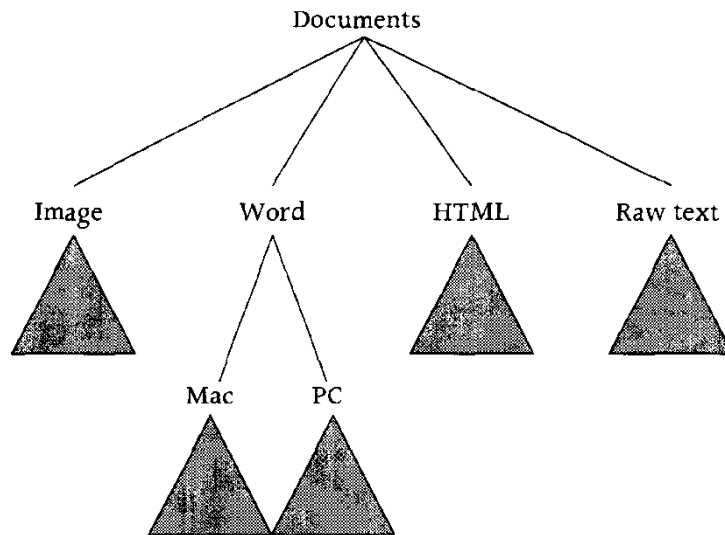


Example of UML Classes



Object Oriented Database Model

- ▶ Each document belongs to a specific class within the system. Some Word documents may belong to multiple classes
- ▶ Each museum can be considered an entity. Different museums may be grouped into one or more categories. A museum may belong to one or multiple categories, and each category may include multiple museums..



Object Oriented Database Model

- ▶ Entities described by attributes and value domains
- ▶ Class inheritance enables shared and specialized attributes
- ▶ Support for heterogeneous attribute sets within a class

Object Oriented Database Model

- ▶ Data is presented in sets
 - ▶ Sets of objects O_{id}
 - ▶ Sets of classes C_{id}
 - ▶ Sets of attributes: Att

Object Oriented Database Model

► Documents

- $O_{id} = \{b_1, b_2, \dots, b_6\}$
- $C_{id} = \{\#html, \#raw\ text, \#image, \#word\ mac, \#word\ pc\}$
- $Att = \{author, date_created, date_modified, key\ words, related_docs\}$

► Museums

- $O_{id} = \{b_{met}, b_{louvre}, \dots, b_{Smithsonian}\}$
- $C_{id} = \{\#art, \#history, \#nature\}$
- $Att = \{address, director, annual_budget, admission_fee, hours, special_exhibits\}$

Object Oriented Database Model

- ▶ An entity is defined by the structure of its attributes and the set of values associated with those attributes.
- ▶ The object-oriented database model allows entities to be described based on common characteristics by grouping them into classes.
 - ▶ It also permits entities to have their own specific attributes.
 - ▶ Two distinct entities belonging to the same class may have completely different sets of attributes and methods

Object Oriented Database Model

► Declaring objects:

Examples:

```
declare b2
```

```
values [author = John Smith;  
       url = http://www.somewhere.com/index.html;  
       date-created= (15, Jan, 1996);  
       date-last-modified=(19, Nov, 1996)]
```

```
declare b3
```

```
values [author = John Smith;  
       url = http://www.somewhereelse.com/index.html;  
       { [link = b1], [link = b2] } ]
```

Object Oriented Database Model

```
declare bSmithsonian  
values [address = "The Mall, Washington DC";  
        director = "Jim Schwartz";  
        special-exhibits = {Caravaggio paintings, Persian  
        manuscripts};  
        affiliated-museums = {bLouvre, bMOMA}]
```

MOMA: Museum of Modern Art, New York.

Object Relationships in Multimedia Systems

- ▶ Link/reference: b3 links to b1 and b2 (hypertext, citations, related media)
- ▶ Composition: strong ownership — deleting a document may delete embedded thumbnail objects
- ▶ Association: weak link — museum affiliated with other museums without ownership
- ▶ Multi-valued attributes: {string}, {Oid_Set} model one-to-many without separate link tables
- ▶ Contrast with relational: relationships are navigable from the object, not only via JOIN queries



Object Oriented Database Model

- ▶ Data type defines the domain for each attribute of objects (Real, Boolean, Integer, Char, String ...)

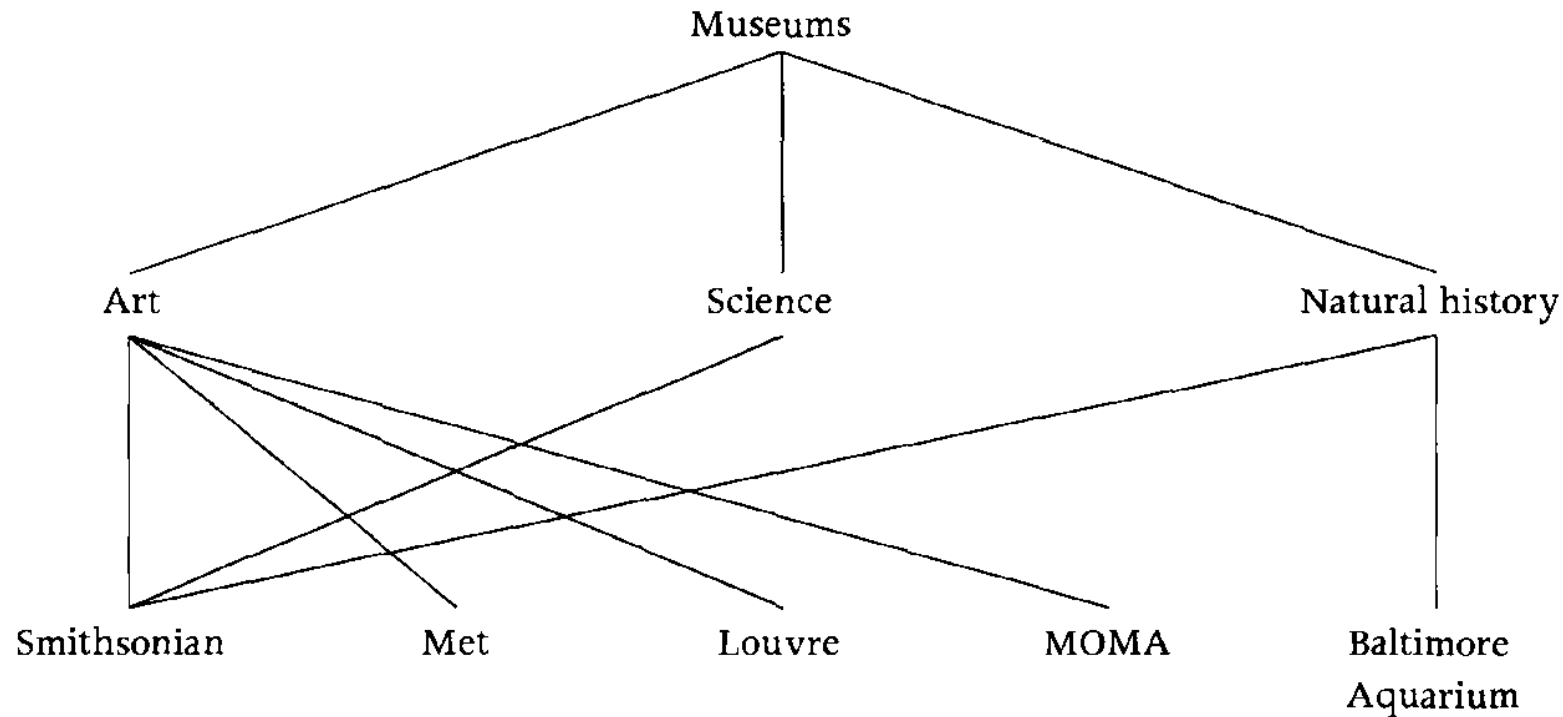
Example:

```
[ author: string;  
  url: urltype;  
  date-created: datatype;  
  date-last-modified: datatype  
  address: string;  
  director: string;  
  special-exhibits: { string }; // biểu diễn một tập các chuỗi  
  affiliated-museums: { Oid_Set } ] // biểu diễn một tập các thực thể
```

Object Oriented Database Model

► Class hierarchy

- In hierarchical class representations, describing entities at each class level does not simply involve listing the data types inherited from higher-level classes to lower-level classes



Object Oriented Database Model

In a class hierarchy, subclasses contain all the attributes of their parent classes, along with additional attributes

Museum	Art (museum)	Smithsonian
address: string; director: string; departments: { string }; budget: real	address: string; director: string; departments: { string }; budget: real ; old-master-collection: { string }; modern-art-collection: { string }; lithograph-collection: { string }; cartographic-collection: { string }	address: string; director: string; departments: { string }; budget: real; old-master-collection: { string }; modern-art-collection-. { string }; lithograph-collection: { string }; cartographic-collection: { string }; animal-collection: { string }; mineral-collection: { string }; aquatic-collection: { string }; airplane-collection: { string }; space-collection: { string }

Object Oriented Database Model

- ▶ In the hierarchical representation described above, **Smithsonian** includes both the *aquatic-collection* and the *airplane-collection* because it is a subclass of both the **Natural History Museum** and the **Science Museum**.

Multiple Inheritance and the Diamond Problem

- ▶ Smithsonian inherits from both Natural History Museum and Science Museum (multiple parents)
- ▶ Attribute inheritance: subclass receives all parent attributes plus its own specialized collections
- ▶ Method inheritance conflict: if two parents define different FindDocs implementations, which runs?
- ▶ Conflict Resolution Policy (CRP) selects: ordering, recency, or priority of parent classes
- ▶ Design guideline: keep hierarchies shallow; document CRP for each multiple-inheritance node



Object Oriented Database Model

▶ Class/Domain Hierarchy

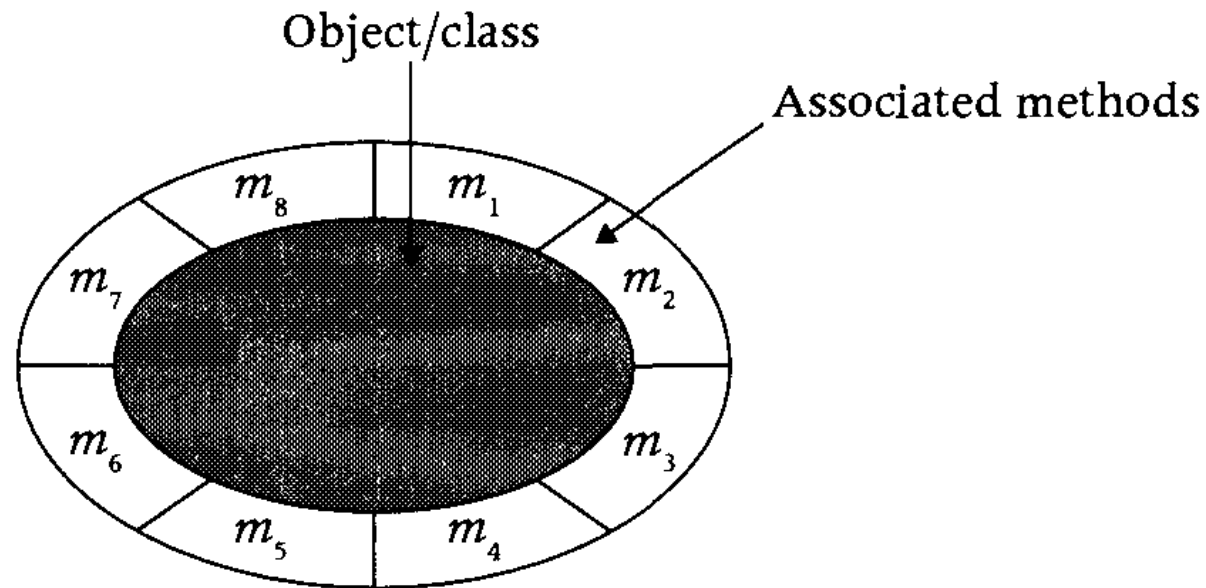
- ▶ The members of a class consist of all its subclasses represented in the classification model.
- ▶ When representing subclasses, it is not necessary to explicitly list all the attributes of the parent class.

- ▶ Example: the *art* class can be represented as follows

```
[ old-master-collection: { string };  
  modern-art-collection: { string };  
  lithograph-collection: { string };  
  cartographic-collection: { string }]
```

Object Oriented Database Model

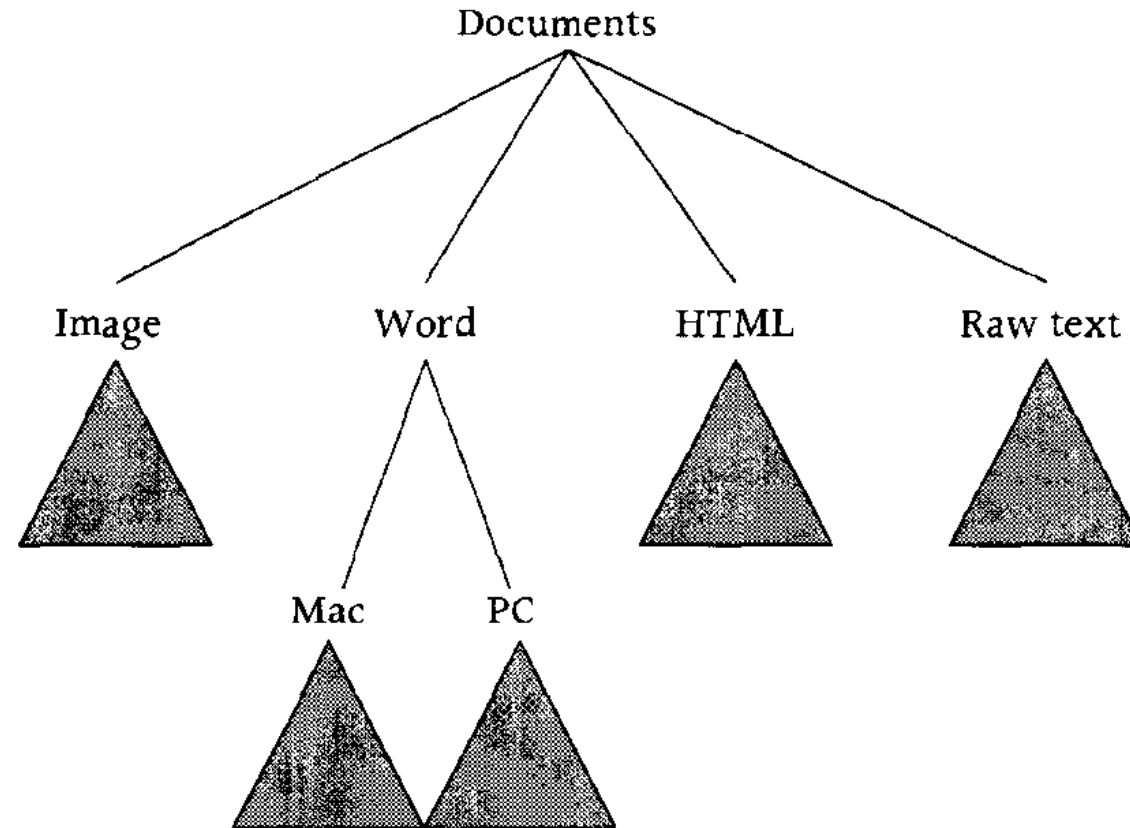
- ▶ Methods: are procedures or operations performed on data classes.
 - ▶ Each entity class is associated with a specific set of methods.
 - ▶ Entities within a given class can only be manipulated or operated on through the methods defined for that class.



Methods and Message Passing in OODBMS

- ▶ Method: named operation with typed inputs/outputs, bound to a class
- ▶ Invocation: `object.message(args)` — the DBMS resolves the correct implementation at runtime
- ▶ Method signature: name + parameter types + return type (must be compatible for inheritance)
- ▶ Constructor/destructor methods: create and remove persistent objects
- ▶ Query methods vs update methods: read-only retrieval (`FindDocs`) vs modify (`UpdateMetadata`)
- ▶ Methods can traverse links: `FindDocs(name)` follows `related_docs` references across object network

Object Oriented Database Model



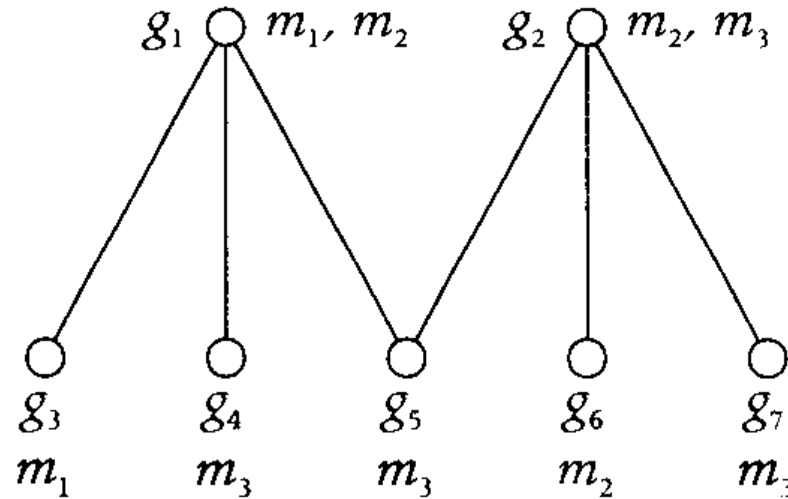
Object Oriented Database Model

- ▶ Example: the FindDocs method takes a person's name (of type *String*) as input and retrieves all documents related to that person.
- ▶ Search methods associated with different classes include:
 - ▶ The Raw_text class provides the FindDocs_d method, which returns related text documents
 - ▶ The Image class provides the FindDocs_i method, which returns related images
 - ▶ The Word class provides FindDocs_w method, which returns related Word documents
 - ▶ For the PC class, there is no corresponding *FindDocs* method defined. In this case, the method inherited from its parent class (FindDocs_w) can be used

Object Oriented Database Model

- ▶ I/O types
 - ▶ Each method associated with an entity class has specific input and output data types
- ▶ A method may be applicable to multiple entity classes. To distinguish a method m when it is applied to an entity class g , it is denoted as m_g
- ▶ Inheritance: a method defined in a class may be inherited and used by the subclasses of that class.
 - ▶ provided that the method signatures are structurally compatible
 - ▶ Conflict resolution: in cases of multiple inheritance, a conflict resolution policy (CRP) is required to determine which method implementation should be applied

Object Oriented Database Model



- Conflict resolution policies:

- Select a parent class (according to a predefined ordering) from a set of multiple parent classes for inheritance
- Select the most recently defined method for inheritance
- Select a parent class based on the highest priority (importance).

Object Definition Language (ODL)

- ▶ ODL declares classes, attributes, relationships, and method signatures (not implementation)
- ▶ Example:
 - ▶ `class Video { attribute title: string; attribute duration: integer;`
 - ▶ `relationship chapters: set<Chapter>;`
 - ▶ `int getDurationInMinutes(); }`
- ▶ Supports collection types: set, list, array of atomic or object types
- ▶ Maps to ODMG standard (legacy) and influences modern OOP persistence APIs



Object Query Language (OQL)

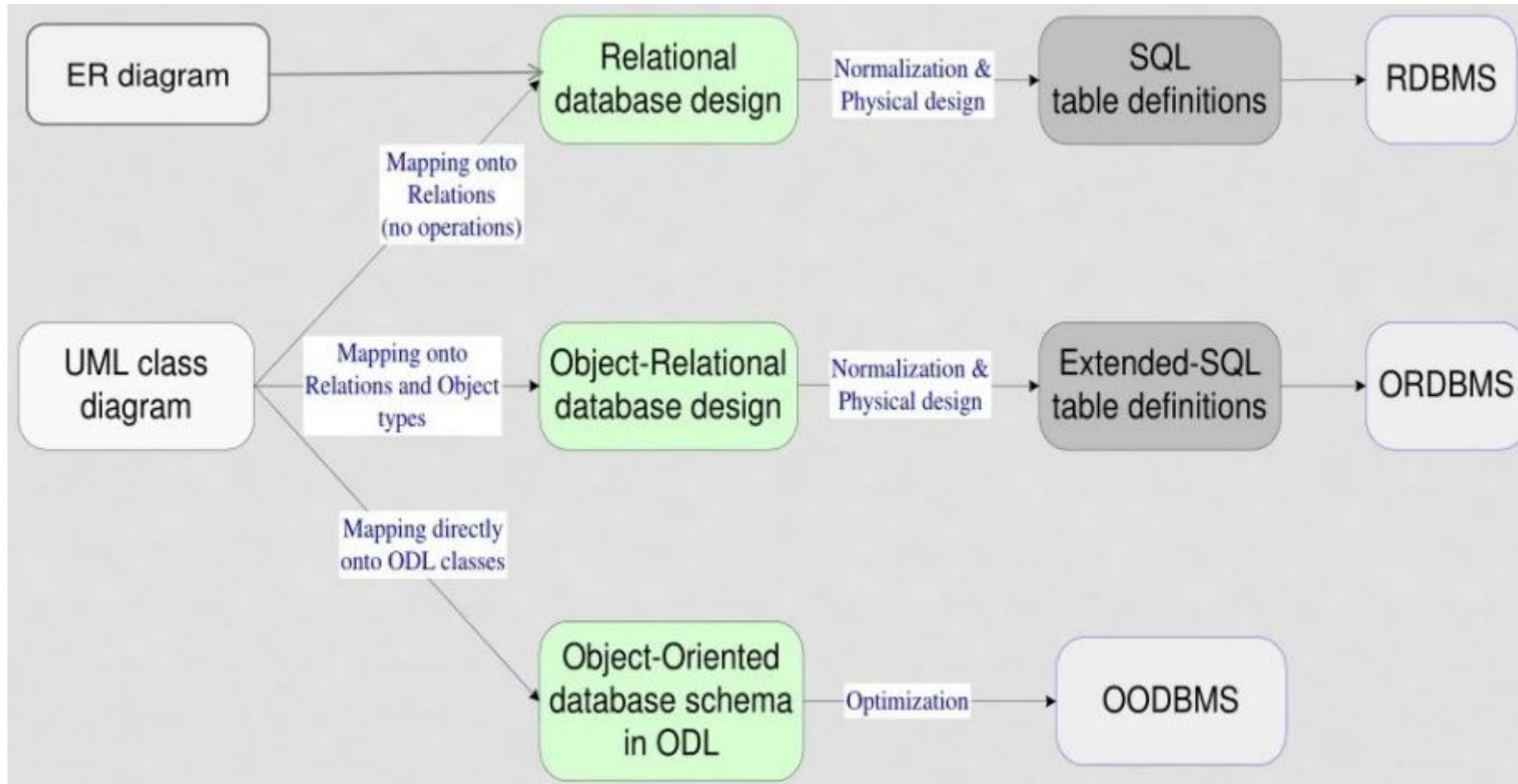
- ▶ OQL extends query capability to object networks with path expressions
- ▶ Example: `SELECT v.title FROM Videos v WHERE v.duration > 3600`
- ▶ Navigate relationships: `SELECT c.title FROM Videos v, v.chapters c WHERE v.title = 'Lecture 5'`
- ▶ Supports object construction, aggregation, and calls to class methods in queries
- ▶ Contrast with SQL: no manual JOIN syntax when relationships are modeled as object references



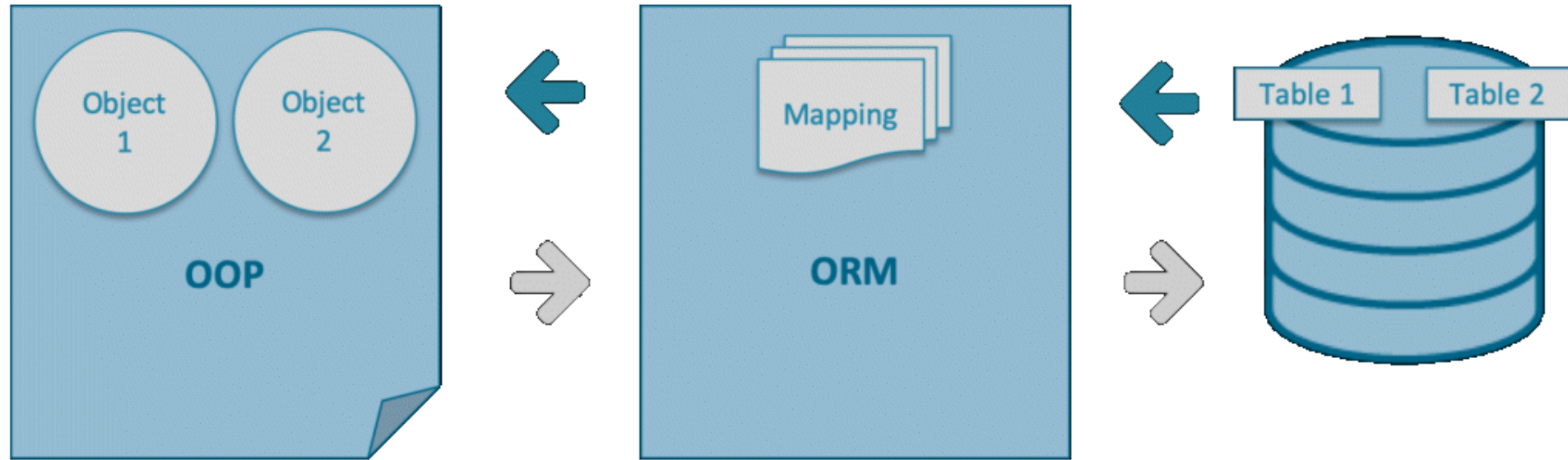
Object Oriented Database Model

- ▶ **Object Definition Language (ODL):** ODL does not specify the executable implementations of methods; instead, it only declares the input and output data types of methods.
- ▶ **Object Query Language (OQL):** In addition to supporting data retrieval based on user queries, OQL also provides mechanisms for accessing and manipulating complex data types.

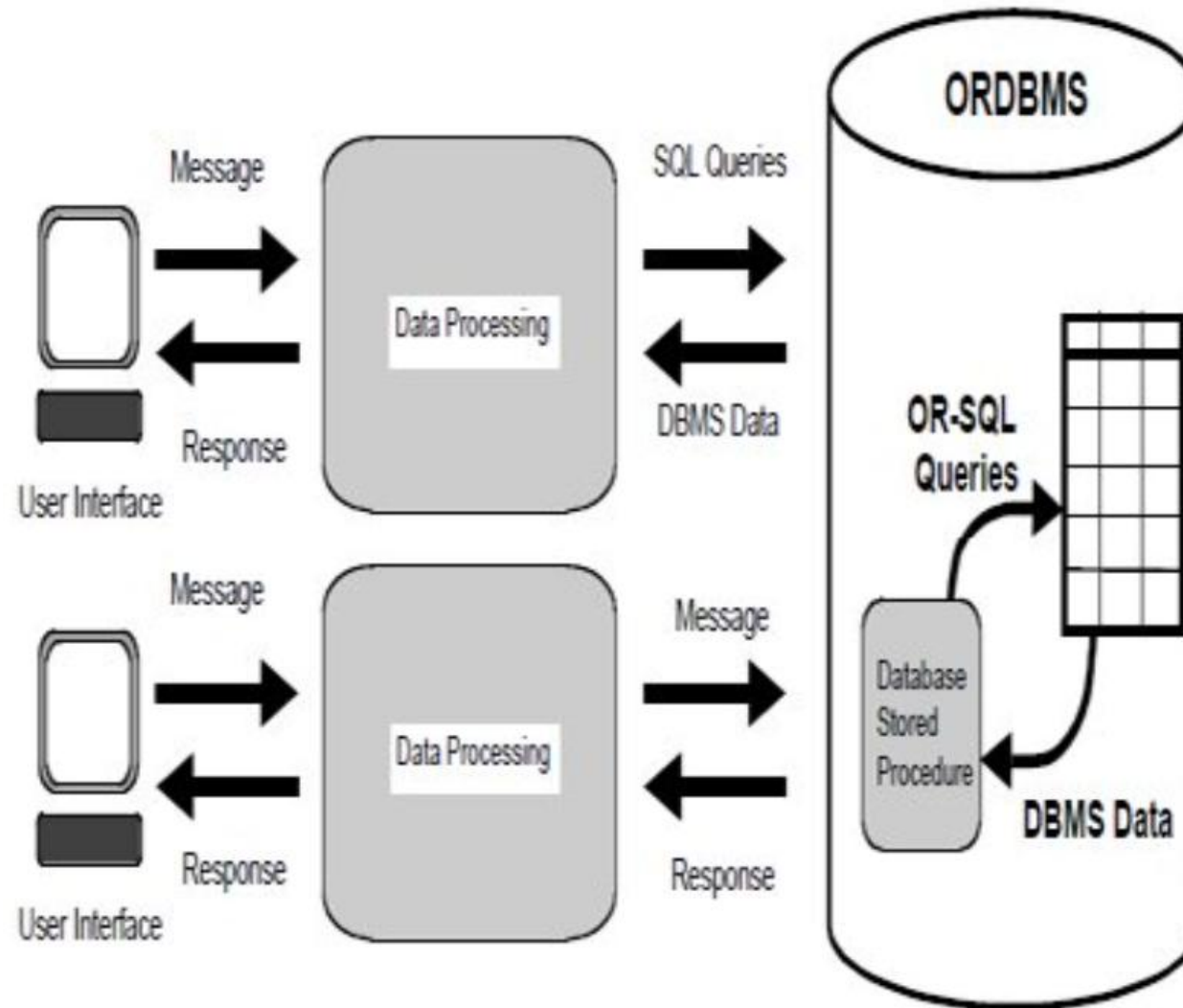
Object-Relational Databases (ORDBMS)



Object-Relational Databases



ORDBMS as the Object Server Architecture



Object-Relational Databases (ORDBMS) — Motivation

- ▶ Goal: keep SQL/relational strengths while adding object features for multimedia
- ▶ Supports user-defined types (UDTs), methods, inheritance, and large objects (BLOB/CLOB)
- ▶ Multimedia fits well: Image, Audio, Video as typed columns with attached operations
- ▶ Examples: PostgreSQL, Oracle, IBM Db2, Informix (historically SQL3 / SQL:1999)
- ▶ Advantage over pure OODBMS: SQL ecosystem, transactions, tools, and team skills remain
- ▶ Advantage over pure RDBMS: nested structures and media-specific methods inside the DB



ORDBMS Core Concepts

- ▶ User-defined types: `CREATE TYPE Image AS (path text, width int, height int, ...)`
- ▶ Typed tables / columns: `photos Image[]`; or column of type `Video`
- ▶ Methods / functions: `extract_thumbnail(img)`, `duration_minutes(vid)`
- ▶ Inheritance: `CREATE TYPE LectureVideo UNDER Video (...)`
- ▶ Large objects: `BYTEA` / `BLOB` for raw media; `TOAST` / `LOB` storage under the hood
- ▶ Collections: arrays, nested tables, composite types for chapters and keyframes



Example: PostgreSQL for Multimedia Metadata

- ▶ CREATE TYPE media_clip AS (title text, duration_sec int, codec text);
- ▶ CREATE TABLE courses (id serial PRIMARY KEY, name text, intro media_clip);
- ▶ CREATE TABLE videos (id serial, course_id int REFERENCES courses(id),
 - ▶ file_path text, features real[], chapters jsonb);
- ▶ Query: SELECT name, (intro).duration_sec FROM courses WHERE (intro).codec = 'h264';
- ▶ JSONB + array types store nested MM structure without many join tables
- ▶ Extensions (PostGIS, pgvector) push ORDBMS further toward multimedia & similarity



Object-Relational vs Pure OO vs Pure Relational

- ▶ Relational: flat tables; JOINS for structure; behavior only in application code
- ▶ OODBMS: native objects/OID navigation; weaker SQL interoperability historically
- ▶ ORDBMS: SQL + UDTs/methods/BLOBs — pragmatic middle ground for MM apps
- ▶ Choose OR when you need SQL reporting plus typed media columns
- ▶ Limitation: complex OO graphs still awkward; polyglot stacks often appear
- ▶ Bridge: OR often coexists with object stores, filesystems, and vector indexes

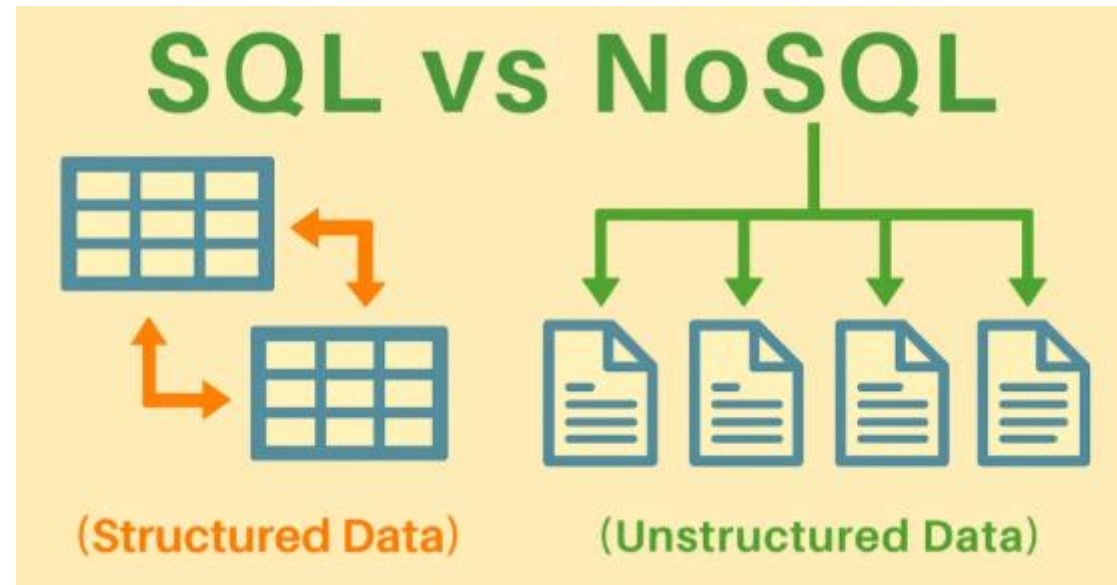


NoSQL Databases — Overview for Multimedia

- ▶ NoSQL: non-relational stores optimized for scale, flexibility, or specific access patterns
- ▶ Main families: document, key-value, wide-column, and (often grouped) graph stores
- ▶ Why MM apps use them: schema evolution, large catalogs, CDN-like object refs, JSON APIs
- ▶ Typical pattern: media files in object storage (S3); metadata/features in NoSQL
- ▶ Trade-off: weaker multi-document ACID vs easier horizontal scale and flexible schemas
- ▶ Examples: MongoDB, CouchDB, Cassandra, DynamoDB, Redis



NoSQL Databases



Document Stores for Multimedia Catalogs

- ▶ One document $\approx$ one logical MM object (video + chapters + tags + thumbnails)
- ▶ Nested arrays/objects avoid join explosion of relational designs
- ▶ Example MongoDB document: {title, duration, speakers[], chapters[{t,label}], tags[]}
- ▶ Queries: find videos with tag 'lecture' and duration > 600; index nested fields
- ▶ Good fit: CMS, media libraries, user-generated content platforms
- ▶ Limits: deep cross-document joins and complex multi-hop relationships



Key-Value and Wide-Column Stores

- ▶ Key-value: `get(key) → blob/metadata`; ideal for session, cache, object ID lookup
- ▶ Redis: fast feature cache, rate limits, queues for ingest pipelines
- ▶ Wide-column (Cassandra/HBase): time-series sensor video metadata, write-heavy logs
- ▶ Partition by `camera_id` / `user_id` for scalable surveillance or streaming analytics
- ▶ Use when access is mostly by primary key or known partition key
- ▶ Not ideal for ad-hoc multi-attribute similarity search alone



Example: Media Catalog in MongoDB

- ▶ Collection videos: store metadata + feature vectors + object-storage URLs
- ▶ `db.videos.createIndex({tags: 1, duration_sec: 1})`
- ▶ `db.videos.find({tags: 'flower', 'chapters.label': 'intro'})`
- ▶ GridFS or external S3 for large binaries; keep only refs in the document
- ▶ Aggregation pipeline: count videos per speaker, average duration per topic
- ▶ Schema versioning: add new fields (`embedding_v2`) without full table migration



Graph Databases — Motivation

- ▶ Multimedia systems are rich in relationships: cites, similar-to, appears-in, derived-from
- ▶ Graph model: nodes (entities) + edges (typed relationships) + properties on both
- ▶ Query pattern: multi-hop navigation ('videos linked to this speaker's talks')
- ▶ Examples: Neo4j, Amazon Neptune, JanusGraph, TigerGraph
- ▶ Strong for recommendation, provenance, knowledge graphs over media archives
- ▶ Complements document/relational stores rather than replacing raw media storage



Graph Model Concepts for Multimedia

- ▶ Nodes: Video, Image, Person, Tag, Scene, Album, Camera
- ▶ Edges: APPEARS_IN, TAGGED_WITH, SIMILAR_TO, PART_OF, DERIVED_FROM, CITES
- ▶ Properties: duration, confidence, timestamp, weight/similarity score
- ▶ Path queries: find all images of people who appear in the same video as X
- ▶ Pattern matching (Cypher/Gremlin/SPARQL) replaces deep JOIN chains
- ▶ Index nodes/edges by label and key properties for performance



Example: Cypher for a Media Knowledge Graph

- ▶ CREATE (v:Video {title:'Lecture 5', duration:3600})
- ▶ CREATE (p:Person {name:'Alice'})-[:SPEAKS_IN {role:'lecturer'}]->(v)
- ▶ CREATE (t:Tag {name:'databases'})<-[:TAGGED_WITH]-(v)
- ▶ MATCH (p:Person {name:'Alice'})-[:SPEAKS_IN]->(v:Video)-[:TAGGED_WITH]->(t)
 - ▶ RETURN v.title, collect(t.name)
- ▶ MATCH path = (v1:Video)-[:SIMILAR_TO*1..2]-(v2) RETURN path LIMIT 20
- ▶ Use case: digital library linking lectures, slides, authors, and related clips



When to Use Graphs for Multimedia

- ▶ Use graphs when relationships are first-class: social tags, provenance, hypermedia
- ▶ Good: 'find related content', influence analysis, scene–object–person networks
- ▶ Weaker alone for bulk storage of BLOBs or pure k-NN on high-dim vectors
- ▶ Hybrid: media blobs in object store; catalog in document DB; links in graph DB
- ▶ Property graphs vs RDF: RDF/SPARQL for linked open data and standards
- ▶ Design tip: keep edge types stable; put volatile scores as edge properties



Vector Databases — Motivation

- ▶ Modern MM retrieval often embeds media into high-dimensional vectors
- ▶ Images, audio, video clips, and text map to embeddings (CLIP, ResNet, Whisper, ...)
- ▶ Query becomes nearest-neighbor search: find top-k vectors closest to the query vector
- ▶ Vector DBs specialize in approximate nearest neighbor (ANN) at large scale
- ▶ Examples: pgvector, Milvus, Pinecone, Weaviate, Qdrant, FAISS (library)
- ▶ Fits content-based image/audio/video retrieval from later course lectures



Vector Data Model and Similarity

- ▶ Record: id + dense vector (dim d) + optional payload metadata (tags, path, type)
- ▶ Distance metrics: cosine, Euclidean (L2), inner product — choose to match training
- ▶ Exact k-NN is $O(N \cdot d)$; ANN indexes trade a little recall for large speedups
- ▶ Common indexes: HNSW, IVF, PQ / product quantization, LSH
- ▶ Hybrid query: filter by metadata THEN rank by vector similarity (or vice versa)
- ▶ Store raw media elsewhere; keep embedding + pointer in the vector index



Example: Image Retrieval with Vectors

- ▶ Offline: for each flower image, compute embedding $e_i \in \mathbb{R}^{512}$; insert (id, e_i , species)
- ▶ Online: embed query image $q \rightarrow e_q$; search top-5 nearest under cosine similarity
- ▶ SQL+pgvector sketch: `SELECT id FROM images ORDER BY embedding <=> :q LIMIT 5`
- ▶ Payload filters: `WHERE species = 'rose'` before or after ANN
- ▶ Evaluation: Precision@k / Recall@k on labeled queries (same as Lecture 1 metrics)
- ▶ This is the data-model layer behind CBIR systems in Lectures 10–12



Vector DB vs Relational / NoSQL / Graph

- ▶ Relational/OR: great for exact attributes; poor native support for ANN at scale
- ▶ Document NoSQL: flexible metadata; vector search needs extensions or side indexes
- ▶ Graph: excellent multi-hop links; not optimized for dense high-dim k-NN alone
- ▶ Vector DB: specialized for similarity; still needs metadata store for rich filters
- ▶ Production MM systems are often polyglot: object store + catalog + vector index
- ▶ Choose by query: exact ID, flexible JSON, relationship path, or 'find similar'



Model Comparison Matrix for Multimedia

- ▶ Relational: ACID SQL; weak nesting/behavior — OLTP metadata, simple catalogs
- ▶ OO: rich objects/methods — tightly integrated MM applications (legacy niche)
- ▶ OR: SQL + UDTs/BLOBs — enterprise MM with reporting needs
- ▶ NoSQL docs/KV: flexible scale — UGC platforms, CMS, media catalogs
- ▶ Graph: relationships — recommendations, provenance, knowledge graphs
- ▶ Vector: similarity — CBIR, semantic search, multimodal retrieval



Polyglot Persistence Architecture for MM Systems

- ▶ Object storage (S3/MinIO): immutable media blobs and derivatives
- ▶ Catalog DB (OR / document): titles, rights, formats, chapter structure
- ▶ Graph (optional): people–media–topic relationships and provenance
- ▶ Vector index: embeddings for content-based retrieval
- ▶ Application API orchestrates filters, graph expansion, then ANN re-ranking
- ▶ Design principle: store each concern where its queries are cheapest



Summary — Multimedia Data Models

- ▶ No single model fits all multimedia workloads
- ▶ Relational → OO → OR: increasing support for structure and behavior
- ▶ NoSQL: schema flexibility and horizontal scale for large catalogs
- ▶ Graph: first-class relationships; Vector: first-class similarity
- ▶ Real systems combine models (polyglot persistence)



Further Reading

- ▶ SQL:1999 / PostgreSQL docs — composite types, JSONB, extensions
- ▶ Sadalage & Fowler — NoSQL Distilled (document, KV, column, graph)
- ▶ Neo4j Cypher manual — graph patterns for media knowledge graphs
- ▶ Milvus / pgvector / FAISS docs — vector indexes and hybrid search
- ▶ Course projects: map your audio/image/video system to this model stack



Exercises and Practice — Overview

- ▶ Exercises 1–6: relational limitations and object-oriented modeling (existing)
- ▶ Exercise 7: object-relational UDTs and SQL methods
- ▶ Exercise 8: NoSQL document design for podcasts
- ▶ Exercise 9: graph schema and multi-hop Cypher queries
- ▶ Exercise 10: vector schema and hybrid similarity search
- ▶ Exercise 11: integrated case study across all six model families

